

RLHF

Preference Learning

Why do we need preference alignment?

Problem with only SFT?

All the methods for the preference alignment?

RL(Reinforcement Learning)Introduction

RL Evaluation timeline

Diff B/W DL & RL

Diff B/W SL & RL

Q-Learning v/s DQN(Deep Q-Network) v/s PPO

What is RLHF?

PPO in RLHF

Practical of RLHF with Custom dataset

Before understanding preference tuning or any of its methods, let's first understand the LLM training pipeline and where preference tuning fits within it.

Pretraining(Knowledge)

(Train the model on a massive dataset to learn general knowledge and language patterns)

↓

Supervised Fine-Tuning (SFT)(Instruction following)

(Instruction-based tuning to teach the model how to follow user instructions)

↓

Preference Tuning / Preference Alignment(Behavior control)

(Preference-based tuning to align model outputs with human preferences)

Methods: RLHF (PPO), GRPO, DPO, ORPO

↓

Aligned LLM (ChatGPT-like behaviour)

Alignment techniques like RLHF, DPO, and ORPO ensure that LLMs not only generate correct outputs but also produce responses that are safe, helpful, and aligned with human preferences.

What is preference alignment?

Preference alignment is the process of training a model to generate outputs that humans prefer.

In simple terms, among many possible answers, it helps the model learn which response humans like more.

Example

Prompt:

"How do I prepare for exams?"

Possible Outputs:

✅ Calm, structured advice → Preferred

"Make a study plan, revise daily, and take proper breaks to stay consistent."

❌ Overly harsh, unsafe advice → Rejected

"If you're not studying 10 hours a day, you're going to fail."

Preference alignment (also called preference learning) is now a critical part of modern LLM training—it is no longer optional.

Why do we need Preference Alignment?

It is essential because while Supervised Fine-Tuning (SFT) improves task performance, it does not ensure that the model's behavior aligns with human values, safety, and expectations.

Modern LLMs are used in real-world systems such as:

- Healthcare assistants
- Finance tools
- Education platforms
- Enterprise copilots

In these environments, behavior matters just as much as correctness.

Problem with pretraining(Predict the next token)

• •

This means the model learns:

- Patterns in data
- Grammar & language structure
- General knowledge

Pretrained models are good at:

- Generating fluent text
- Answering general questions

But the training data may contain:

- Incorrect or misleading information
- Toxic or biased language
- Conflicting opinions
- Unsafe or harmful responses

But NOT good at:

- Following instructions properly
- Being safe and aligned
- Giving consistently correct or helpful answers

“Pretraining makes the model knowledgeable, but not controllable and trustworthy”

Problem with only SFT (Supervised Fine-Tuning)

"SFT teaches what is good, but it doesn't teach what is better or what is wrong."

Supervised Fine-Tuning teaches the model:

- How to follow instructions
- The expected format of answers
- Task-specific knowledge
- It improves capability.

SFT teaches:

- "This is a good answer"

But it does NOT explicitly say:

- "This answer is bad, avoid it"

What SFT does not guarantee

SFT does not explicitly teach:

- Which answers are better than others
- Which answers are unacceptable
- Human values like politeness, safety, tone, or helpfulness

So during SFT:

- Probability of good answers increases
- Probability of bad answers can also increase accidentally

We do preference alignment to ensure the model is:

- ✔ Helpful
- ✔ Harmless
- ✔ Honest
- ✔ Polite
- ✔ Safe
- ✔ Non-toxicity
- ✔ Consistency with human expectations

In short:

Preference alignment bridges the gap between “knows language” and “behaves well”.

What all methods are there for this?

What all methods are there for this?				
Algorithm	Uses RL?	Reward Model?	Reference Model?	Complexity
PPO	✔	✔	✔	Very High
GRPO	✔	✔	✔	High
DPO	✘	✘	✔	Medium
ORPO	✘	✘	✘	Low

A. RL-based methods

RLHF (Reinforcement Learning from Human Feedback)

Pipeline:

1. Collect human preference data
2. Train a reward model
3. Optimize the LLM using reinforcement learning

Common RL algorithms

- PPO (Proximal Policy Optimization) traditional method
- GRPO (Group Relative Policy Optimization) advanced / optimized version

PPO → single response per reward

GRPO → multiple responses compare karke relative reward deta hai

Characteristics

Very powerful

Strong safety alignment

Complex

Expensive

Needs multiple models

B. Non-RL-based methods

DPO (Direct Preference Optimization)

Key idea:

"Make chosen answers more likely than rejected ones."

What it needs

- Preference data (chosen vs rejected)
- Reference model (frozen)
- No reward model
- No RL loop

Benefits

- ✓ Simple
- ✓ Stable
- ✓ Cheaper than RLHF

C. Single-stage preference alignment (Modern methods)

These methods merge SFT + preference alignment.

ORPO (Odds Ratio Preference Optimization)

Key idea:

- Strongly reward preferred answers
- Weakly penalize rejected answers
- Do both in the same loss

What ORPO removes

No RL

No reward model

No reference model

Benefits

- ✓ One training stage
- ✓ Very fast
- ✓ Low memory
- ✓ Clean pipeline

RLHF(PPO) and GRPO

- Complex
- Expensive
- Multiple models needed: Base model , Reward model , RL training loop

DPO (MODERN ALTERNATIVE)

Directly train model:

Chosen > Rejected

ORPO (LATEST METHOD)

SFT + Preference learning in one loss

Complete AI Tree & where RL exists in AI umbrella?

AI Hierarchy (Clean + Industry Ready)

1. Artificial Intelligence (AI)

Top-level umbrella

- Goal: Machines that can mimic human intelligence

2. Machine Learning (ML)

Way to achieve AI

- Learns from data instead using stats methods

3. Core ML Paradigms

Supervised Learning: Learn from labeled data

Unsupervised Learning: Learn patterns without labels

4. Reinforcement Learning Branch

"System jo environment se learn kare aur intelligent decisions le"

Classical RL

- Q-Learning
- SARSA

Works on small problems (tables)

5. Deep Learning (DL)

using Neural Networks

6. Neural Network Architectures

ANN (MLP): Basic neural network

CNN: Used in Computer Vision

RNN / LSTM: Used in sequence data ((NLP))(before transformers)

GAN (Generative Adversarial Networks)

- Generative models
- Used for image, video, data generation

Deep Reinforcement Learning

RL + Neural Networks

- DQN
- PPO

Solves complex real-world problems

7. Transformers (Breakthrough)

Key architecture in modern NLP

- Attention mechanism
- Handles long context

8. Large Language Models (LLMs)

Built using Transformers

- GPT
- LLaMA
- Claude

9. Generative AI (GenAI)

LLMs + generative capability

- Text generation
- Image generation
- Code generation

10. Agentic AI

Next level AI systems

- Planning
- Tool usage
- Multi-step reasoning

Autonomous decision-making

What is RL?

Reinforcement Learning (RL) is a learning method where a model learns through trial and error.

It is a way of building intelligent AI systems.

An agent learns by interacting with its environment and receiving rewards or penalties.

Over time, it learns to choose the best possible actions.

"Reinforcement Learning is about learning optimal behavior through rewards and penalties."

1. Agent

👉 The learner (model / AI)

2. Environment

👉 Where the agent acts (game, real world, chatbot, etc.)

3. Action

👉 What the agent does

4. State

👉 Current situation

5. Reward

👉 Feedback (good or bad)

State → Action → Environment → Reward → Update → Repeat

Simple Example (Game)

👉 Imagine a game:

- Agent = Player
- Action = Move left/right
- Reward:
 - +10 → win
 - -10 → lose

👉 Over time:

Agent learns → which actions give maximum reward



This example explains Reinforcement Learning using a simple real-life analogy. Now let's understand it programmatically with a coding example.

👉 The whole idea of:

Agent → Action → Reward → Learn

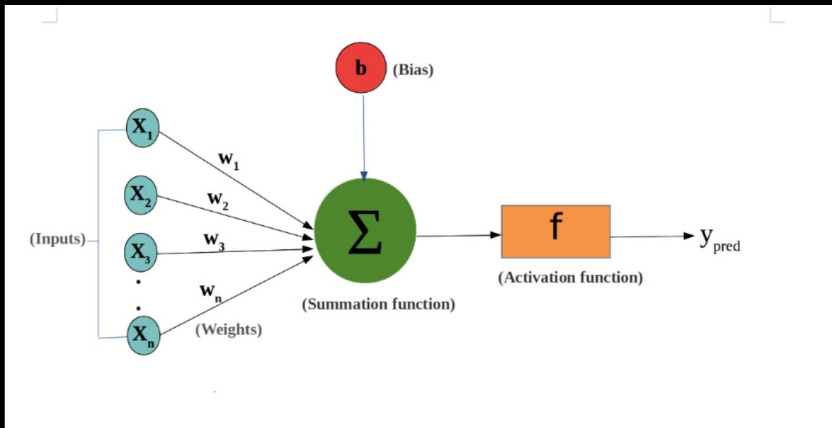
can be applied anywhere.

"Take action → get feedback → improve decision over time"

RL Evolution Timeline
RL evolved from tables → neural networks → human-aligned intelligence.”

Phase	Description
1. 1950s – Foundations (MDP)	• Reinforcement Learning is based on Markov Decision Process (MDP) • Core idea: State → Action → Reward → Next State This is still the backbone of RL today
2. 1980s–1990s – Classical RL	• Key algorithms introduced: ○ Q-Learning ○ Temporal Difference (TD Learning) ✓ Learning using tables Not scalable to large problems
3. 2000s – Slow Progress	• Mostly theoretical work • Limited real-world applications Reason: • Low compute power • No deep learning support
4. 2013–2015 – Deep Reinforcement Learning (Breakthrough)	• Introduction of Deep Q-Network (DQN) ✓ Neural networks + RL ✓ Solved Atari games This is where RL became practical
5. 2016 – Policy Gradient & Actor-Critic	• Shift from value-based → policy-based learning ✓ Instead of Q-values, we directly learn: $\pi(a s)$
6. 2017 – PPO (Most Important Phase)	• Proximal Policy Optimization (PPO) introduced ✓ Stable training ✓ Controlled updates ✓ Widely used in industry This is the core RL algorithm used in many systems
7. 2018–2020 – RL in Real Systems	• Used in: ○ Robotics ○ Game AI (AlphaGo, AlphaZero) ✓ Complex decision-making tasks
8. 2020–2022 – RL + LLM (RLHF)	• Reinforcement Learning applied to LLMs RLHF (Reinforcement Learning with Human Feedback) Pipeline: Pretraining → SFT → Reward Model → PPO ✓ Used to train ChatGPT-like models

What is a deep Learning?



What is Deep RL?

Deep RL $\rightarrow$ RL + neural networks

Deep Learning learns patterns from data using neural networks, Reinforcement Learning learns optimal actions using reward signals, and Deep Reinforcement Learning combines both to solve complex decision-making problems using neural networks.

Deep RL $\rightarrow$ RL powered by neural networks

AlphaGo (Google)

ChatGPT (IMPORTANT)

DL = modelling approach

RL = learning paradigm

Difference between Deep Learning and Reinforcement Learning(Deep RL)			
Concept	Classical RL	Deep Learning (DL)	Deep Reinforcement Learning (Deep RL)
Input	State (discrete)	Input data (image, text, etc.)	State (high-dimensional, continuous)
Output	Action	Prediction	Action
Core Idea	Q-table / Policy table	Neural network learning	RL + Neural Network
Feedback Signal	Reward	Loss	Reward-based loss
Objective	Maximize cumulative reward	Minimize loss	Maximize reward
Learning Type	Trial & Error	Supervised / Unsupervised	Trial & Error + NN
Representation	Tabular (Q-table)	Neural Network	Neural Network (Q(s,a), Policy)
Scalability	❌ Low	✅ High	✅ High
Generalization	❌ No	✅ Yes	✅ Yes
Training Process	Update Q-table	Forward → Loss → Backprop	Forward → Reward → Loss → Backprop
Algorithms	Q-learning, SARSA	CNN, RNN, Transformers	DQN, PPO, RLHF
Data Requirement	Environment interaction	Dataset (labeled/unlabeled)	Environment interaction
Use Case	Simple games, small problems	Vision, NLP	Robotics, Games, LLM alignment

Concept	Supervised Learning	Reinforcement Learning
Data	Labeled data	No labels
Feedback	Immediate (correct/incorrect)	Delayed reward
Goal	Predict correct output	Maximize reward
Learning Style	Direct learning	Trial & Error
Example	Image classification	Game playing / ChatGPT alignment

Difference between Supervised Learning and Reinforcement Learning

"Supervised learning optimizes against correct answers, while reinforcement learning optimizes based on rewards without knowing the correct answer."

Supervised Learning = Teacher present

RL = No teacher, only reward system

1. Supervised Learning (Simple Form)

Objective (Simple)

"Loss" = "Average Error between Prediction and True Answer"

Update Rule (Simple)

$\theta = \theta - \text{"learning rate"} \times \text{"gradient"}$

Meaning:

- Agar error zyada hai → weights change karo
- Goal = prediction $\approx$ correct answer

2. Reinforcement Learning (Simple Form)

Objective (Simple)

"Goal" = "Maximize Total Reward"

Mathematical simple form:

$J(\theta) = \text{"avg"}(r_1 + r_2 + r_3 + \dots)$

Notation (Simple Meaning)

- s = state (current situation)
- a = action (decision)
- $\pi_\theta(a|s)$ = probability of taking action a in state s
- r = reward (feedback)
- θ = model parameters
- $J(\theta)$ = total reward

Update Rule (Simple)

"Increase probability of good actions"

Slightly mathematical but simple:

"Update" $\propto$ "Reward" $\times \log\left[\frac{f_0}{f_1}\right]$ ("probability of action")

Meaning:

Agar reward high:

same action fir se karne ki probability badhao

Agar reward low:

us action ko avoid karo

What is "learning" in RL

Forget theory. Think like this:

Learning = changing behaviour based on feedback

What actually changes?

In ML / DL: Learning = updating parameters (weights)

In RL

Model has: Policy $\pi(a | s)$

Meaning: Probability of taking action "a" in state "s"

Learning = change this probability:

Good action → probability ↑

Bad action → probability ↓

What is Q-learning?

Q-Learning is a value-based Reinforcement Learning algorithm

Tu ek robot hai

👉 Tu ek grid me move karta hai

[S][][]

[][][G]

- S = Start
- G = Goal
- Actions = {up, down, left, right}

Q-table ka matlab

👉 Har state + action ka score store karna

Example table:

State Left Right Up Down

S 0 0 0 0

A 0 0 0 0

👉 Matlab:

"Agar main state S me hu aur RIGHT jaun → kitna reward milega?"

Table kaise fill hota hai?

Robot try karta hai actions (trial & error)

Example:

👉 State S → Right → next state → reward = +10

Update:

$$Q(S, \text{Right}) = 0 + \alpha(10 + \gamma * \text{future} - 0)$$

End me kya hota hai?

Table ban jata hai:

State Left Right Up Down

S 1 9 0 2

👉 Robot sikhta hai:

"S me ho → RIGHT best hai"

Q-Learning Formula (MOST IMPORTANT)

$$\text{New } Q = \text{Old } Q + \text{Learning} \times (\text{Target} - \text{Old } Q)$$

Where:

$$\text{Target} = r + \gamma \max Q(s', a')$$

Current knowledge → $Q(s, a)$

You take action → get reward r

Then look at best future → $\max Q(s', a')$

Update your belief

Step 3: Problem (Real World)

Q-Table Explosion Problem

For small problems, Q-learning works fine:

```
Q = {  
  "left": 0,  
  "right": 0  
}
```

This is manageable.

Ab soch:

👉 State = image (car camera)

👉 1 image = 1000×1000 pixels

👉 Possible states = billions 🤖

❌ Q-table ka issue

- Har state ke liye row chahiye
- Har action ke liye column

👉 Impossible to store

Step 4: Neural Network entry (DQN)

Ab magic samajh 🔥

👉 Table banane ke bajaye:

“Ek function bana dete hain jo Q value predict kare”

🔥 Neural Network kya karta hai?

Instead of:

Q-table lookup

We do:

$Q(s,a) = \text{Neural Network}(s)$

🎯 Example

Input:

👉 Image (state)

Output:

👉 Q values for actions

Input: Car camera image

Output:

Left = 2.1

Right = 5.6

Brake = 3.0

👉 Model bolta hai:

“Right best hai”

🧠 Important Shift (MOST IMPORTANT)

Q-learning DQN

Table store karta hai Function learn karta hai

Memory heavy Generalize karta hai

Exact values Approximate values

🔥 Real intuition

👉 Q-learning:

“Har situation yaad rakho”

👉 DQN:

“Pattern samjho aur guess karo”

PPO ka core idea (1 line)
🔥 PPO = "safe learning"
Model ko update karo lekin zyada change mat karo

🔥 Problem kya tha?
DQN / Q-learning me:
• Value learn karte the
But PPO me:
🔥 Direct action lena seekhte hain

🔥 PPO ka flow (step-by-step)
Step 1: Actor action leta hai
🔥 Neural network (Actor) bolta hai:

$\pi(a|s)$
Example:
• Left → 20%
• Right → 70%
• Brake → 10%
🔥 Action pick kiya: Right

Step 2: Environment se reward
🔥 Car ne right liya → reward mila:
• Good move → +10
• Bad move → -5

Step 3: Critic judge karta hai
🔥 Critic bolta hai:
"Ye state kitni achhi thi?"

$V(s)$
Step 4: Advantage calculate hota hai
🔥 Simple idea:

Advantage = Reality - Expectation
• Reality = reward mila
• Expectation = critic ka value
🔥 Agar:
• Positive → good action 🔥
• Negative → bad action 📉

🔥 Step 5: PPO ka MAGIC (CLIPPING)
🔥 PPO bolta hai:
"Update kar, but limit ke andar"
Mathematically:

$$\text{"ratio"} = (\pi_{\text{"new"}}(a|s)) / (\pi_{\text{"old"}}(a|s))$$

🔥 PPO loss:

$$\min_{\theta} \mathbb{E}_{s \sim \mathcal{D}} [\text{"ratio"} \odot A^{\pi_{\theta}} \text{ clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) - A]$$

1. Q-Learning (Tabular)
🔥 Type: Value-Based
🔥 Storage: Table
• Stores: $Q(s, a)$ in table
• Works only for small problems
❌ Problem:
• State space bada → table explode

2. DQN (Deep Q Network)
🔥 Type: Value-Based
🔥 Upgrade of Q-learning
• Replaces table with Neural Network:

$Q(s| \theta, a| \theta, \theta)$

✅ Solves:
• Large state space (images, games, etc.)
⚠️ Still limited:
• Mostly works for discrete actions only

3. PPO (Proximal Policy Optimization)
🔥 Type: Policy-Based (actually Actor-Critic)
• Directly learns:

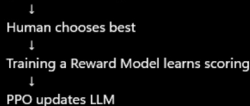
$\pi(a|s)$

(what action to take)
✅ Advantages:
• Works for continuous actions (robotics, control)
• More stable training than older RL methods
• Used in RLHF (ChatGPT training 🔥)

What is RLHF?

RLHF is a method or that uses reinforcement learning algorithms like PPO, along with human feedback, to align language models with human preferences.

Prompt → LLM → Multiple Answers



SFT: "This is the correct answer"

RLHF: "This answer is better than the other one"

Step 1: LLM generates text

Model gives 2 answers:

Answer A:

AI is the ability of machines to learn from data and make decisions like humans.

Answer B:

AI is something computers do with many algorithms and complex functions.

Step 2: Human preference → Reward Model

Human says:

- Answer A → 🟡 Better (clear, simple)
- Answer B → 🟠 Worse (confusing)

What happens now?

Reward Model learns:

- A → +2 reward
- B → -1 reward

Meaning:

Reward Model becomes the judge

Step 3: PPO updates LLM

Now PPO's job:

- Output like A → increase probability
- Output like B → decrease probability

Next time the model will try:

- Give simple, clear answers
- Avoid confusing answers

RL vs RLHF

RL is a learning paradigm using rewards,

RLHF is a framework that uses RL + human feedback to align models.

PPO vs RLHF

PPO is an algorithm,

RLHF is a framework where PPO is used.

What is Reward Modelling?

Reward Modeling is the process of training a model that can score or rank outputs based on human preferences.

In RLHF: Reward Model = Human preference ka proxy (judge)

WHY Reward Modeling Needed?

LLM ko directly yeh nahi pata hota:

- kya "better" hai
- kya "safe" hai
- kya "clear" hai

supervised learning me:

Correct answer milta hai ✅

real world me:

Multiple correct answers hote hain ❌

Example

Prompt:

Explain AI

Outputs:

- A → simple, clear
- B → complex, confusing

🟡 dono technically correct ho sakte hain

🟡 but:

Human prefers A

Reward Modeling Pipeline

Step 1: Generate multiple responses

Prompt → LLM → multiple outputs

Step 2: Human labeling (VERY IMPORTANT)

Human compares:

A > B

dataset banega:

(prompt, chosen, rejected)

Step 3: Train Reward Model

input:

(prompt + response)

output:

score (scalar value)

Objective

Reward(chosen) > Reward(rejected)

Core Idea

instead of:

correct vs incorrect

we learn:

better vs worse

Now lets understand how PPO works in RLHF?

Real Example (LLM / ChatGPT)

User asks:

"Explain AI"

Model outputs:

- Response A → human likes 🍌 (+ reward)
- Response B → bad 🍌 (- reward)

PPO:

- Increase probability of A
- Decrease probability of B
- But slowly (clipping)

Without PPO:

- ✗ Training unstable
- ✗ Model forgets previous learning
- ✗ Huge jumps in behavior

With PPO:

- ✓ Stable
- ✓ Smooth learning
- ✓ Works in real systems

PPO is used in the last stage to align the model with human preferences.

or

PPO is used to: "Update the LLM so it generates responses humans prefer"

PPO in RLHF is like:

"If humans like this answer → say it more often

If humans dislike it → avoid it

But don't overreact suddenly"

Model weights (neural network)

NOT:

- ✗ No Q-table
- ✗ No manual rules
- ✓ Only probabilities of words change

Why PPO was used in ChatGPT

- ✓ Stable training
- ✓ Works with large models
- ✓ Handles human feedback

Step 1: Start with SFT Model

- You already have a trained model (after supervised fine-tuning)

Example:

User: Explain AI

Model: AI is a field of study...

Step 2: Generate Multiple Responses

Model generates different outputs:

Response A: Clean explanation ✓

Response B: Confusing ✗

Step 3: Reward Model gives score

- 🍌 A separate model (Reward Model) evaluates:

Response Score

A +2

B -1

- 🍌 This score = reward

Step 4: PPO Update Starts

Now PPO comes into play 🍌

🌟 Inside PPO (RLHF Context)

Step 4.1: Old vs New Policy

- Old policy = current LLM
- New policy = updated LLM

- 🍌 Compare probabilities of generated text

Step 4.2: Compute Advantage

- 🍌 Advantage = "how good this response is"

Good response → positive advantage

Bad response → negative advantage

Step 4.3: Update Model (IMPORTANT)

- 🍌 PPO does:

- Increase probability of good responses
- Decrease probability of bad responses

BUT...

- 🍌 With clipping:

Don't change too much at once

Model → generates response → gets REWARD → updates policy using PPO

⚡ Core RL Loop (Hidden inside RLHF)

Classic RL:

State → Action → Reward → Update Policy

- ✓ Same thing in RLHF:

RL Concept RLHF Equivalent

State Prompt (user input)

Action Generated response

Reward Score from Reward Model

Policy LLM

- 🍌 So:

Prompt → Response → Reward → PPO Update

- 🍌 THIS is Reinforcement Learning

Why it doesn't "look like RL"

You're confused because:

- 🍌 There is:

- No environment like game
- No agent moving physically

But actually:

- 🍌 The LLM itself is the agent
- 🍌 The environment = prompt + reward model

